Mohamed bin Zayed University of Artificial Intelligence

AI1215: Introduction to Machine Learning

Course Project, Summer 2026

PitRossa

Fuel-Corrected Tyre Degradation Modelling and Pit-Stop Decision Support for Formula 1

Team name: Anti-Ferrari Disasterclass

Members: Abdelmonaim Bounite

Project title: PitRossa: Tyre Degradation Modelling and Pit-Stop Decision Support for Formula 1

Chosen topic: Custom topic (motorsport). Predicting Formula 1 tyre degradation from public timing and weather data, and converting it into pit-stop decision support. Supervised regression with an applied decision layer.

Data: FastF1 Python API, 2022 to 2025 seasons. Models: scikit-learn.

June 26, 2026

Notation: t: lap index; k: lap number; N: race laps; $s(t)$: stint of lap t; L_t: observed lap time; F_t: fuel mass; a_t: tyre age (`TyreLife`); c: compound; $\tilde{L}_t$: fuel-corrected lap time; D_t: `DegradationDelta` (target); $\hat{D}_t$: prediction.

1 Introduction and Problem Formulation

1.1 What is Formula 1 and Why Do Pit Stops Matter?

Formula 1 (F1) is the fastest and most prestigious car racing sport in the world. A race lasts about 90 minutes. During the race, the tyres on the cars are extremely important. As tyres get old and wear out, they lose grip on the track, and the car becomes much slower. To fix this, drivers enter the pit lane to get a fresh set of tyres.

However, driving through the pit lane takes a lot of time, usually about 20 to 25 seconds. Teams must perfectly balance this time loss against the speed they will gain back with new tyres. Stopping just one lap too early or too late can easily lose a race.

Figure 1: Scuderia Ferrari on track, the team that I support.

1.2 Strategy Problems, Undercuts, and Overcuts

Deciding the exact moment to pit is very difficult. Scuderia Ferrari is a famous team known for building incredibly fast cars, but they are historically known for making poor strategy choices. They frequently call their drivers into the pits at the wrong time. This project's motivation and our team name reflect this exact historical issue (Figure 2).

F1 strategy involves two major tactical moves:

- **The Undercut:** Pitting before your rival. You use the fast speed of your brand new tyres to jump ahead of them while they are still driving slowly on old tyres.
- **The Overcut:** Staying out longer than your rival. If the rival's new tyres take too long to warm up, or if they get stuck behind slow traffic, you can jump ahead of them by driving fast in clear air before pitting later.

1.3 The Hidden Factor: Aerodynamics and Dirty Air

Another huge factor in strategy is aerodynamics. F1 cars use the air to push the car down into the track (downforce). However, as a car cuts through the air, it leaves behind a messy trail of turbulent wind called dirty air (Figure 3). If a driver follows closely behind another car, this dirty air removes their downforce. The car slides more, which destroys the tyres much faster. Teams must plan their pit stops to avoid putting their driver back onto the track in the middle

(a) The Ferrari pit wall experience.

(b) Strategy recruitment process.

Figure 2: Internet culture frequently highlights Ferrari's strategic missteps. Building an AI to prevent these exact mistakes is the core motivation behind this project.

of dirty air.

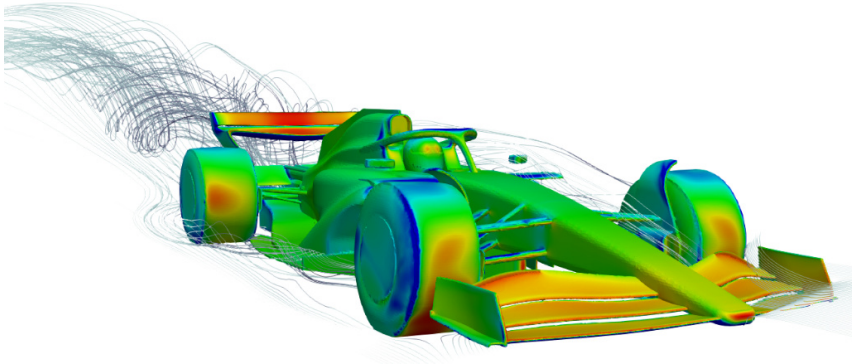


Figure 3: Aerodynamic wake of a 2022 F1 car. Following closely in this turbulent dirty air heavily increases tyre wear. Image pulled from Taylor (2022).

Currently, teams rely on slow computer programs to guess the best strategy before the race (Thomas et al., 2025). This project, called **PitRossa**, builds a fast machine learning model to read live data and help teams make better pit stop decisions on the fly.

1.4 Data Collection and Overview

We collected data for the 2022 to 2025 seasons using the FastF1 Python API (Oehrly, 2025). The data includes lap times, tyre types, track position, and weather. F1 teams keep their fuel amounts a secret, so we calculate the fuel load ourselves by assuming cars burn fuel steadily. The final dataset has **61,185 laps across 69 races** and 53 starting columns.

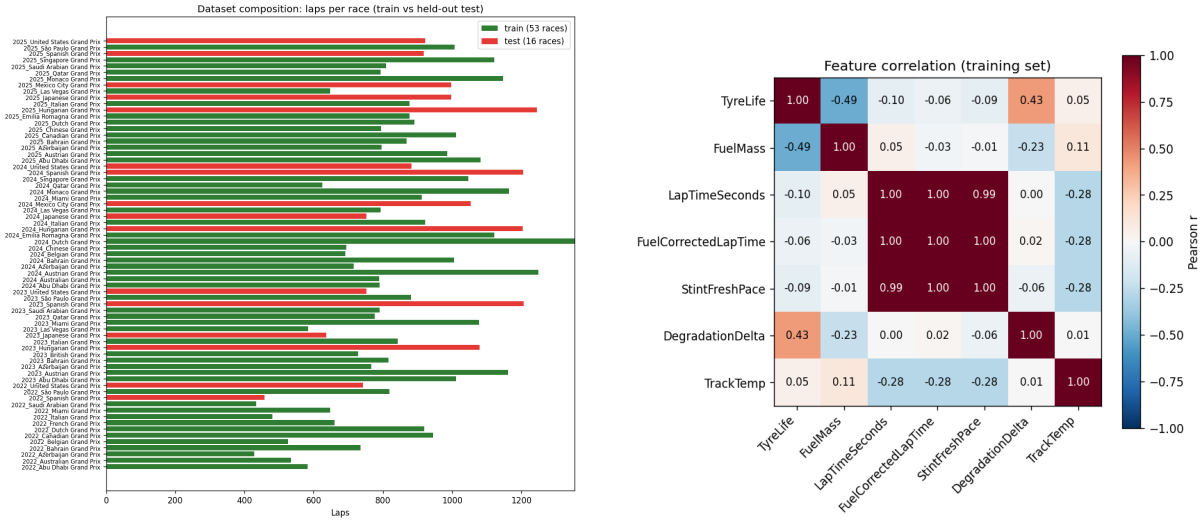
2 Preprocessing and Feature Engineering

2.1 Removing the Fuel Effect

When a car burns fuel, it gets lighter and faster. When tyres wear out, the car gets slower. Because both happen together, raw lap times are confusing. We first regress lap time on fuel mass to isolate the car's true pace:

$$L_t = \alpha_r + \gamma_r F_t + \varepsilon_t \qquad \text{(ordinary least squares, fitted per race } r\text{)} \tag{1}$$

$$\tilde{L}_t = L_t - \gamma_r F_t, \qquad F_t = F_0\left(1 - \frac{k_t - 1}{N - 1}\right), \quad F_0 = 109\,\text{kg} \tag{2}$$



(a) Sample composition by team and race.

(b) Feature correlations.

Figure 4: Data overview. Tyre age and fuel mass are perfectly linked within a stint. We must remove the fuel effect to see true tyre wear.

2.2 Creating the Target Variable

Absolute lap times depend heavily on the specific race track. We created a new target variable called `DegradationDelta` by subtracting the best fresh tyre pace from the fuel-corrected lap time:

$$P_s = \text{median}\left(3 \text{ smallest } \tilde{L}_t : s(t) = s\right), \qquad D_t = \tilde{L}_t - P_{s(t)} \tag{3}$$

The additive constant left by the zero-fuel projection is absorbed by P_s, so the fuel-correction reference is irrelevant to D_t. Our target is now near 0 seconds when the tyre is new, and it grows as the tyre wears out.

3 Model Training and Selection

3.1 Comparing the Models and Ensembles

We split our data by entire circuits to test how the model performs on unseen tracks. We trained five individual models and two ensemble methods (Voting and Stacking) to predict the `DegradationDelta`.

$$\mathbf{x}_t = \left(a_t,\ a_t^2,\ \mathbf{1}[c_t{=}\text{M}],\ \mathbf{1}[c_t{=}\text{S}],\ \text{Temp}_t,\ \{\mathbf{1}[\text{team}_t{=}j]\}_j\right), \qquad \hat{D}_t = \beta_0 + \boldsymbol{\beta}^\top \mathbf{x}_t \tag{4}$$

$$\boldsymbol{\beta} = \arg\min_{\boldsymbol{\beta}} \sum_{t \in \text{train}} \left(D_t - \beta_0 - \boldsymbol{\beta}^\top \mathbf{x}_t\right)^2, \qquad \frac{\partial \hat{D}}{\partial a} = \beta_a + 2\beta_{a^2}\, a \tag{5}$$

The baseline simply predicts the mean tyre wear. We use Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) to measure performance:

$$\hat{D}^{\,0} = \frac{1}{|\text{train}|} \sum_{t \in \text{train}} D_t, \qquad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n} \left|D_i - \hat{D}_i\right|, \qquad \text{RMSE} = \sqrt{\tfrac{1}{n}\textstyle\sum_{i=1}^{n}(D_i - \hat{D}_i)^2} \tag{6}$$

$$\text{margin} = \frac{\text{MAE}_0 - \text{MAE}_{\text{model}}}{\text{MAE}_0}, \qquad \text{(by-circuit split)}\ \ \text{circuits(train)} \cap \text{circuits(test)} = \varnothing \tag{7}$$

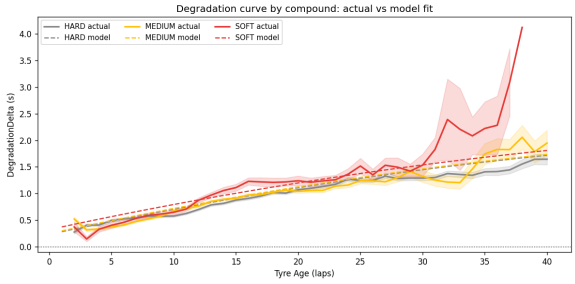
We also tested ensemble models, combining our basic models to see if they could improve performance:

$$\hat{D}^{\text{vote}} = \frac{1}{M}\sum_{m=1}^{M}\hat{D}^{(m)}, \qquad \hat{D}^{\text{stack}} = g\big(\hat{D}^{(1)},\dots,\hat{D}^{(M)}\big), \quad g \text{ linear on out-of-fold predictions} \tag{8}$$

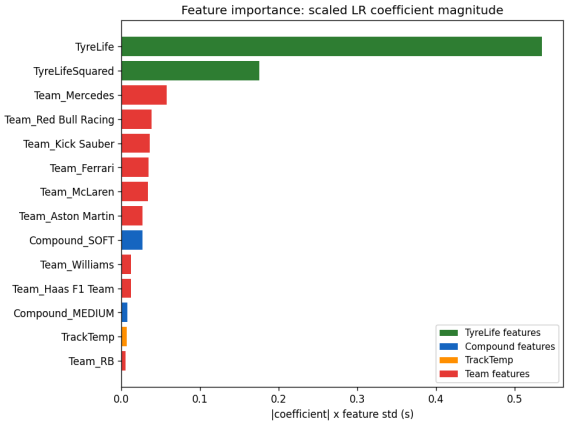
Model	Test MAE (s)	Train MAE (s)
Baseline (predict mean)	0.638	0.633
Linear regression	**0.534**	0.545
Decision tree (depth 3)	0.537	0.548
Random forest (200)	0.636	0.219
XGBoost (300)	0.565	0.503
Voting (mean)	0.550	0.439
Stacking (linear meta)	**0.533**	0.565

Table 1: Performance on unseen tracks. Stacking performed marginally better, but Linear Regression was chosen as the primary model because its coefficients are fast and easily interpreted for our strategy tool.

The complex models (like Random Forest) memorize the training tracks but fail on new tracks. While Stacking achieved a slightly better Test MAE (0.533s), Linear Regression (0.534s) was selected because it is incredibly fast and its coefficients can be read directly by our strategy math.



(a) Actual vs predicted tyre wear.



(b) Linear regression features.

Figure 5: The model correctly shows that softer tyres wear out faster than harder tyres.

4 From Prediction to a Pit Decision

4.1 Framing the Strategy (MDP)

We built a tool that decides to "Stay Out" or "Pit." We frame this decision as a Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}), \quad \text{horizon } N, \text{ discount } 1, \qquad s_k = (a_k, c, \dots), \quad u_k \in \{\underbrace{0}_{\text{stay}}, \underbrace{1}_{\text{pit}}\} \tag{9}$$

$$a_{k+1} = \begin{cases} a_k + 1, & u_k = 0 \\ 0, & u_k = 1 \end{cases} \qquad L_k = \underbrace{\rho + \gamma F_k}_{\text{common to both actions}} + D(a_k, c) + P_{\text{loss}}\,\mathbf{1}[\text{outlap}] \tag{10}$$

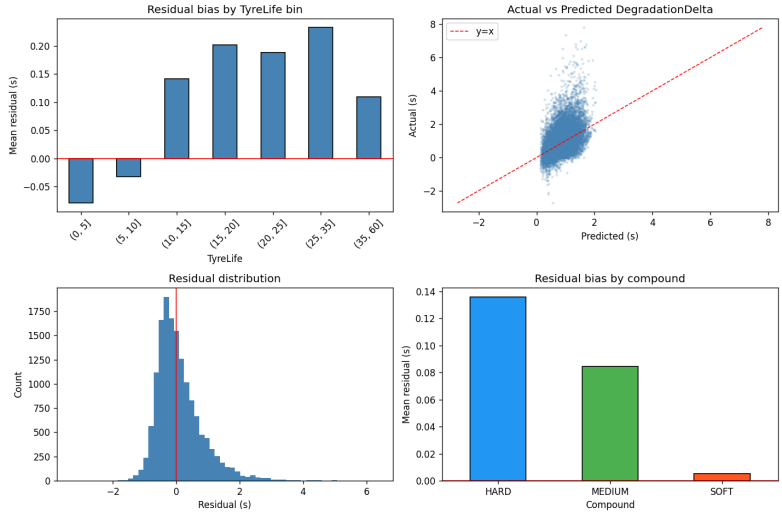


Figure 6: Error checks. The model makes small errors on very old tyres because it misses the sudden cliff where tyres completely lose grip.

$$\min\ J = \sum_{k=1}^{N} L_k, \qquad V_k(s) = \min_{u \in \mathcal{A}} \Big[L_k(s,u) + V_{k+1}\big(f(s,u)\big) \Big], \quad V_{N+1} \equiv 0 \tag{11}$$

4.2 The Greedy Saving Tool

We use a greedy one-step approximation to optimize time saving. If staying out costs more time than getting new tyres (including the pit loss), the tool says it is time to pit:

$$C_{\text{stay}} = \sum_{j=0}^{N-k_0} D(a_0 + j,\, c) \tag{12}$$

$$C_{\text{pit}}(k) = \sum_{j=0}^{(k-k_0)-1} D(a_0 + j,\, c) \;+\; P_{\text{loss}} \;+\; \sum_{j=0}^{N-k} D(j,\, c) \tag{13}$$

$$\Delta(k) = C_{\text{stay}} - C_{\text{pit}}(k), \qquad k^\star = \arg\max_k \Delta(k) \quad (\text{pit iff } \Delta(k^\star) > 0) \tag{14}$$

The break-even statement (pit now, with $R = N - k_0 + 1$ laps left) proves the decision rides purely on the degradation shape:

$$\text{pit now} \iff \sum_{j=0}^{R-1} \big[D(a_0 + j,\, c) - D(j,\, c) \big] > P_{\text{loss}} \tag{15}$$

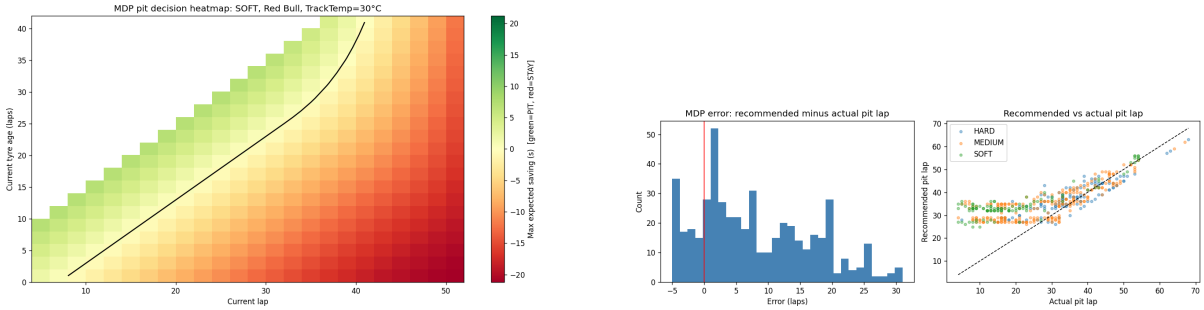
4.3 Uncertainty Bands and Live Updates (POMDP)

Predicting the future always has errors. We calculate an uncertainty band to show the risk:

$$\text{SD}\big(\Delta(k)\big) = \sigma\sqrt{R}, \qquad 90\%\ \text{interval: } \Delta(k) \pm z_{0.95}\,\sigma\sqrt{R}, \quad z_{0.95} = 1.645 \tag{16}$$

Sometimes a tyre wears out faster than expected. We built a Belief-state POMDP that watches the live lap times. As the driver completes laps, it learns the real wear rate and corrects the math on the fly. Let $g(a) := \hat{D}(a, c, \dots)$ be the predicted shape and σ be the residual standard deviation:

$$D_t = \beta\, g(a_t) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2), \qquad \beta \sim \mathcal{N}(\mu_0, \tau_0^2) \;\; (\mu_0{=}1.03,\ \tau_0{=}0.54) \tag{17}$$



(a) Stay Out vs Pit map. **(b)** Model vs Real F1 Teams.

Figure 7: The tool tells the driver when to pit. It stops later than real teams because it does not worry about traffic or other drivers.

$$\tau_n^{-2} = \tau_0^{-2} + \sigma^{-2}\sum_{i=1}^{n} g(a_i)^2, \qquad \mu_n = \tau_n^2\left(\mu_0\,\tau_0^{-2} + \sigma^{-2}\sum_{i=1}^{n} g(a_i)\,D_i\right) \tag{18}$$

$$\beta \mid D_{1:n} \sim \mathcal{N}(\mu_n, \tau_n^2), \qquad \text{predictive at age } a: \;\; \mathcal{N}\big(\mu_n g(a),\; g(a)^2\tau_n^2 + \sigma^2\big) \tag{19}$$

$$\mathrm{Var}\big(\Delta(k)\big) = \underbrace{\sum_{\text{stay}}\big[g(a)^2\tau_n^2 + \sigma^2\big]}_{\text{current set, learned rate}} + \underbrace{\sum_{\text{fresh}}\big[g(a)^2\tau_0^2 + \sigma^2\big]}_{\text{new set, prior rate}}, \qquad \Delta(k) \pm 1.645\sqrt{\mathrm{Var}\big(\Delta(k)\big)} \tag{20}$$

As $n \to \infty$, $\tau_n^2 \to 0$, so the band collapses to $\sigma\sqrt{R}$ and the fixed band is recovered as the fully learned limit of the POMDP. This gives the race engineers a perfectly updating, live mathematical tool.

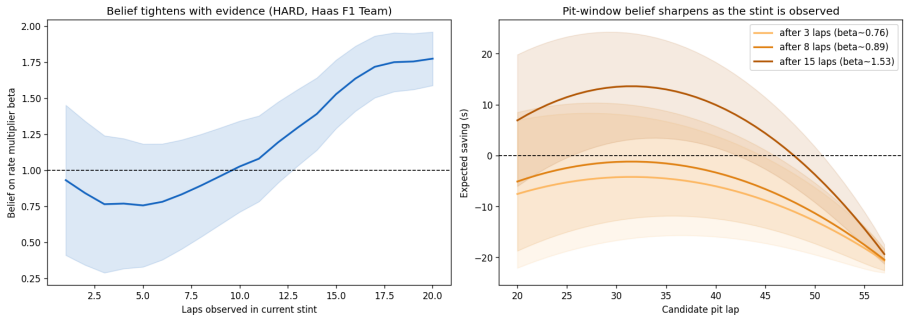


Figure 8: Live updates. As the car drives more laps, the tool becomes more confident about exactly when the tyre will die.

5 Conclusions and Future Work

By creating the `DegradationDelta` feature, PitRossa isolates true tyre wear. Linear Regression generalizes well and allows us to build a mathematically tight, live strategy MDP tool that F1 teams can use without relying on heavy simulations.

Limitations & Future Work: We had to estimate fuel loads. Our tool also focuses purely on optimizing one car. In real life, teams execute undercuts and overcuts to avoid dirty air, which requires pitting earlier than pure physics demands. Future work must integrate aerodynamic wake penalties and multi-agent game theory to handle traffic.

Academic Integrity & AI Usage Statement: AI tools were used to assist in the writing and proofreading of this report, generating the README file, organizing thoughts, providing

conceptual guidance, and assist with the mathematical correctness of the implementation. All final modelling decisions, code, and analytical findings were exclusively authored and verified by the student.

References

Cappello, C. and Hoegh, A. (2025) *A State-Space Approach to Modeling Tire Degradation in Formula 1 Racing*. arXiv preprint arXiv:2512.00640.

Fieni, G., Neumann, M.-P., Furia, F., Caucino, A., Cerofolini, A., Ravaglioli, V. and Onder, C. H. (2025) "Game Theory in Formula 1: From Physical to Strategic Interactions", arXiv preprint arXiv:2503.05421.

Fieni, G., Wüthrich, J., Neumann, M.-P. and Onder, C. H. (2026) "Learning-based Multi-agent Race Strategies in Formula 1", arXiv preprint arXiv:2602.23056.

Oehrly, T. (2025) *FastF1: A Python package for accessing and analysing Formula 1 timing data*, version 3.6.1. Available at: https://github.com/theOehrly/FastF1

Pedregosa, F., Varoquaux, G., Gramfort, A. et al. (2011) "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, 12, pp. 2825-2830.

Taylor, M. (2022) *Aerodynamic Studies of a 2022 F1 Car*. Available at: https://maxtayloraero.com/2022/01/17/aerodynamic-studies-of-a-2022-f1-car/

Thomas, D., Jiang, J., Kori, A., Russo, A., Winkler, S., Sale, S., McMillan, J., Belardinelli, F. and Rago, A. (2025) "Explainable Reinforcement Learning for Formula One Race Strategy", in *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing (SAC '25)*. New York: ACM.